



BreatheSafe

Forecasting Delhi's air , so you can plan before you breathe

Team 4 · Data Engineering Project · 5.8M sensor readings

An LSTM-powered air-quality forecaster built on a full data pipeline

What We Found in the Raw Data

Before
engineering

The dataset arrived as one 5.8M-row parquet file : messy, unstructured, and not ready for any analysis.



19.4M missing cells

15% of all data was empty and entire weather columns half-blank



1 column 100% null

vws_m_s had zero values across all 5.8M rows



Duplicate columns

timestamp duplicated datetime; station duplicated station_id



13 pollutants, not 8

5 unexpected VOCs mixed in with the standard CPCB pollutants

Raw data quality: 84.9% complete · No schema · No structure · Not query-ready

How We Cleaned & Structured It

Before
engineering

84.9%
before



89.4%
after cleaning

+4.6 points

Action	Why	Impact
Dropped 3 columns	100% null + 2 duplicates	22 → 19 cols
Filled missing values	Station-month median (respects season)	1.86M values filled
Checked negatives/dupes	Data integrity	0 found (clean)
Hive partitioning	Fast access by month	24 partition files

Why station-month median, not overall mean? January and July weather in Delhi differ hugely a global mean would inject false values. The median per station per month respects seasonal patterns.

Technique 1 - Storage & Database Choice

Design

Why DuckDB (columnar) over a row-based database? We proved it with a benchmark.

Metric	SQLite (row)	DuckDB (columnar)
Load time	71.5 s	11.7 s
File size	1,078 MB	80.5 MB
Query: avg PM2.5/month	1,852 ms	87 ms
Query: count/pollutant	7,051 ms	164 ms

21–43x

faster queries

13.4x

smaller file

Why columnar wins

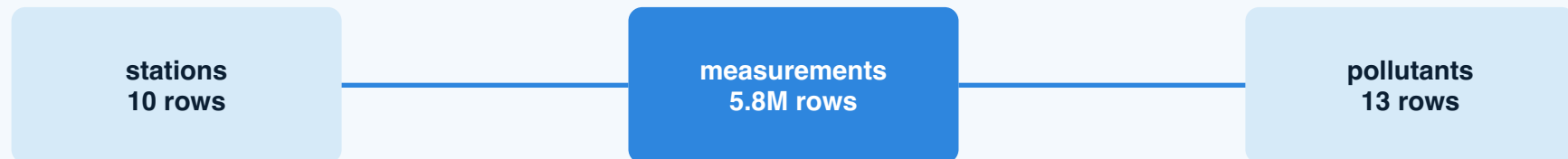
A query like "average PM2.5" only needs 2 of 22 columns. DuckDB reads just those 2 and skips the rest. SQLite reads every row whole all 22 columns every time. On big analytical queries, that's the entire difference.

Also chose Parquet over CSV: 51.8 MB vs ~700 MB, types preserved, far faster reads.

Technique 2 - Data Model & Transformation

Design

Star schema — one fact table, two dimension tables



"Delhi" stored 10x not 5.8Mx :normalization keeps the fact table lean.

Three transformation patterns applied

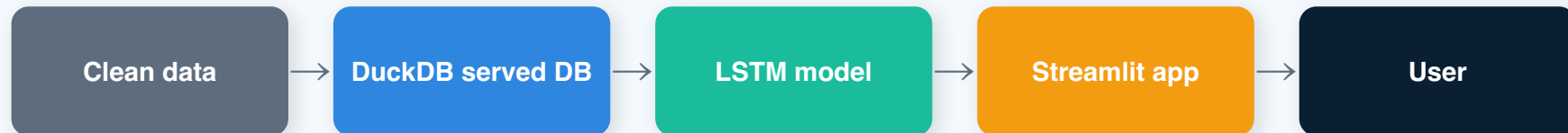
Batch
Processed 24 monthly chunks 5.8M → 312 rows 2.28 sec

MapReduce
628K PM2.5 rows Map → Shuffle → Reduce 50 keys

ETL
Extract → Transform → Load 3 output tables 4.13 sec

Technique 3 - Serving & Forecasting Engine

Design



Serving = data as a product

Clean + transformed data loaded into a served DuckDB with 5 ready-to-query tables. Downstream users (our app) connect and query instantly i.e no re-cleaning, no re-processing.

The forecasting model

An LSTM deep-learning model trained on 2 years of hourly PM2.5. Uses the past 48 hours to predict the next 24 like a weather forecast, but for pollution.

Model performance

Architecture: 2-layer LSTM (64 → 32 units) + dropout · **Input:** past 48 hours · **Output:** next 24 hours

Forecast accuracy (MAE): [FILL IN YOUR MAE FROM TRAINING] $\mu\text{g}/\text{m}^3$

Prototype — BreatheSafe Live Demo



Meet Nimbus : your air guide



Best & worst hours

See the 24-hour forecast and know exactly when to go out



Safest zone right now

Compares all 10 stations and points to the cleanest area



Nimbus reacts

The cloud changes mood — calm when safe, alarmed when hazardous



Switch to the live app

What Makes Us Different

Novelty



We forecast and others report

Most air-quality tools tell you how bad it is RIGHT NOW. We predict the next 24 hours so people can plan ahead. Prediction, not just description.



Real deep learning, end to end

An LSTM trained on 2 years of hourly data not a lookup table. The full journey from 5.8M raw rows to a live neural-network forecast.



Built to be used daily

Nimbus the cloud mascot makes air quality friendly and approachable designed for everyday people, not data scientists.

Only 2 teams ran a database benchmark — we were one of them.

Contributions

Full pipeline ownership — who built what

Component	Owner
Data ingestion & partitioning	
Data cleaning & EDA	
Database design & DuckDB benchmark	
Transformation (Batch / MapReduce / ETL)	
LSTM forecasting model	
Streamlit app & deployment	
Presentation & slides	
Testing & documentation	